UC3 dominates — 70% of all calls come from checking task status. UC4 and UC5 are a different problem: not too many calls, but **too much data**. `task.stdout` dumps 50MB when you need 10 lines. There's no "give me just the artifact names" without iterating everything. The fix for those is bounded access, not fewer calls.

Technical Approach

Architecture: Three Layers

	What	Effect
Layer 1	Extension Package	Smart functions using `filter_tasks_by_metadata` + bounded iteration. No core changes. Works on service $\geq$ 2.5.0.
Layer 2	Metadata Service	Add `_limit`, `_order` to endpoints (DB already supports it). ~150 lines. Benefits everyone, not just agents.
Layer 3	Client API	`ServiceMetadataProvider`: auto-use new query params (version-gated). Transparent speedup. Existing code gets faster.

Extension Package: `metaflow-agent-client`

Uses the `metaflow_extensions` mechanism (no core changes needed):

```
from metaflow_extensions.agent_client import find_failures,
    get_recent_runs

# 4 HTTP calls instead of 56
result = find_failures("ForeachFlow/8")
# {"failures": ["ForeachFlow/8/process/126"], "total_tasks": 50}

# Bounded listing instead of fetching all history
recent = get_recent_runs("ForeachFlow", limit=5, status="failed")
```

Six utility functions, each with internal routing:

- `find_failures()` — uses `filter_tasks_by_metadata("attempt_ok", "False")`
- `get_recent_runs()` — bounded listing with optional status filter
- `batch_run_status()` — batch status for recent runs
- `run_summary()` — structured overview of steps, tasks, failures
- `tail_logs()` — last N lines of a task's stdout/stderr
- `get_runs_since()` — time-filtered listing

Metadata Service Changes (~150 lines)

Change	Lines	What it enables
Add `_limit` to listing endpoints	~15	"Last 5 runs" without fetching all
Add `_order` to listing endpoints	~15	Sort by timestamp, run number
Simplified status via `attempt_ok`	~40	Task status without artifact fetching
Cross-task artifact listing	~40	Artifact metadata across tasks in one call
Bounded log endpoint	~40	Last N lines, no full download

All changes wire to existing `find_records()` infrastructure. The DB layer already supports these parameters.

ServiceMetadataProvider Enhancement

Follows the established version-gating pattern (same as `_supports_attempt_gets` at `service.py:260`):

```
if cls._supports_query_params is None:
    version = cls._version(None)
    cls._supports_query_params = (
        version is not None
        and version_parse(version) >= version_parse("X.Y.Z")
    )
# When True: append ?_limit=N&_order=-ts_epoch to listing URLs
# When False: current behavior (fetch all, filter client-side)
```

Backward Compatibility

- **Old client + new service**: Old clients don't send query params. Service returns all results. Safe.
- **New client + old service**: Version check detects old service, skips query params. Safe.
- **No UI Backend**: All functions have fallback paths. Slower, but correct.
- **Existing code**: `Flow()`, `Run()`, `Task()` behavior unchanged.

Timeline

Phase	Goal	h	Milestone
1 (Wk 1–4)	Extension package: 6 utility functions, unit + integration tests	90	Midterm checkpoint
2 (Wk 5–8)	Metadata service `_limit`/`_order` + provider version-gating	90	Core changes ready
3 (Wk 9–12)	Agent simulation (5 scenarios), MCP tool schemas	90	End-to-end validated
4 (Wk 13–16)	User + agent docs, PR prep per CONTRIBUTING.md, final report	80	Submitted

Midterm checkpoint (Week 4): Working extension package with 6 functions, all tested, benchmark showing improvement over the naive path. Each phase delivers standalone value — Phase 1 alone is a complete, usable deliverable.

Risks: Core Runtime changes require pre-approved issues per CONTRIBUTING.md — Phase 1 (extension package) requires no core changes and delivers full value independently. Status SQL complexity is mitigated by using the simplified `attempt_ok` check (3 lines, 90% coverage). Two-repo coordination is handled via version-gated detection.

Commitments & Disclosure

Availability: 22+ hours/week (350h ÷ 16 weeks). No other employment during GSoC. Exam periods and vacations will be disclosed to the mentor in advance.

AI disclosure: AI tools were used for codebase exploration, benchmark scaffolding, and document drafting. All code, analysis, and architectural decisions are my own. I can explain every line and design choice independently.